# Fonctions, Defer, Pointeurs et Mémoire

---

## 1. Fonctions Avancées en Go

En Go, les fonctions sont des citoyens de première classe (*first-class citizens*) : elles possèdent un type propre, peuvent être stockées dans des variables, passées en argument à d'autres fonctions (callbacks, middlewares) ou retournées par une autre fonction.

### 1.1 Retours multiples et valeurs de retour nommées

Contrairement au C où retourner plusieurs valeurs oblige à passer des pointeurs en paramètres (`void f(int in, int *out1, int *out2)`) ou à créer une `struct` dédiée, une fonction Go peut retourner nativement plusieurs valeurs sur la pile. C'est le socle de toute la gestion des erreurs en Go.

```go
package main

import "fmt"

// 1. Retour multiple standard (le plus courant en Go)
func diviserEntier(a, b int) (int, int, bool) {
	if b == 0 {
		return 0, 0, false
	}
	return a / b, a % b, true
}

// 2. Valeurs de retour nommées (initialisées automatiquement à leur Zero Value !)
func calculerStats(valeurs []float64) (somme float64, moyenne float64) {
	for _, v := range valeurs {
		somme += v
	}
	if len(valeurs) > 0 {
		moyenne = somme / float64(len(valeurs))
	}
	return // "Naked return" : retourne automatiquement les variables 'somme' et 'moyenne'
}
```

*Bonne pratique en entreprise :* Les valeurs de retour nommées sont excellentes pour documenter la signature d'une fonction ou pour modifier une valeur de retour depuis un bloc

`defer` (voir section 2.3). En revanche, évitez le `return` nu (*naked return*) dans les fonctions de plus de 5 lignes, car il nuit à la lisibilité.

## 1.2 Fonctions variadiques (`...T`)

Une fonction variadique accepte un nombre arbitraire d'arguments (de `0` à $N$) pour son dernier paramètre. Sous le capot, le compilateur convertit ces arguments en une **Slice `[]T`**.

```Go
// 'nombres' est reçu comme une slice de type []int
func additionner(prefixe string, nombres ...int) {
	total := 0
	for _, n := range nombres {
		total += n
	}
	fmt.Printf("%s : %d (reçu %d éléments)\n", prefixe, total, len(nombres))
}

func main() {
	additionner("Vide")          // nombres vaut nil (len = 0)
	additionner("Direct", 1, 2, 3) // nombres vaut []int{1, 2, 3}

	// Dépaquetage (Unpacking) d'une slice existante avec l'opérateur '...' :
	maSlice := []int{10, 20, 30}
	additionner("Depuis Slice", maSlice...) // Passe directement la slice sans la recopier !
}
```

## 1.3 Fonctions anonymes et Closures (Fermetures lexicales)

Une **Closure** est une fonction anonyme qui capture et conserve une référence vers les variables de son environnement extérieur, même après que la fonction englobante a terminé son exécution.

```Go
// Générateur de séquence (Factory) : retourne une fonction de type 'func() int'
func creerCompteur(depart int) func() int {
	compteur := depart // Cette variable s'échappe sur le tas (Heap) car elle est capturée !

	return func() int {
		compteur++ // Modifie la variable capturée à chaque appel
		return compteur
	}
}
```

```
func main() {
	c1 := creerCompteur(0)
	c2 := creerCompteur(100)

	fmt.Println(c1()) // 1
	fmt.Println(c1()) // 2
	fmt.Println(c2()) // 101 (c1 et c2 ont chacun leur propre état mémoire
isolé)
}
```

---

## 2. Le mot-clé `defer` : Exécution différée et pièges subtils

En C, la gestion de la libération des ressources (fermer un fichier `fclose`, libérer un pointeur `free`, déverrouiller un `pthread_mutex_unlock`) dans une fonction comportant plusieurs points de sortie (`return`) est un cauchemar qui oblige souvent à utiliser le pattern `goto cleanup;`.

En Go, l'instruction **`defer`** enregistre un appel de fonction pour qu'il soit exécuté **automatiquement à l'instant exact où la fonction englobante se termine** (que ce soit par un `return` normal ou lors d'un crash `panic`).

### 2.1 Règle n°1 : Ordre d'exécution LIFO (Pile *Last-In, First-Out*)

Lorsqu'une fonction contient plusieurs instructions `defer`, elles sont empilées. Le **dernier** `defer` enregistré sera le **premier** exécuté à la sortie de la fonction. C'est logique : si la ressource B dépend de la ressource A ouverte avant elle, il faut fermer B avant de fermer A.

```
Go
func demoLIFO() {
	defer fmt.Println("1. Fermeture connexion Base de Données")
	defer fmt.Println("2. Fermeture Transaction")
	defer fmt.Println("3. Fin de la requête")
	fmt.Println("--- Exécution du traitement métier ---")
}
// Affichage :
// --- Exécution du traitement métier ---
// 3. Fin de la requête
// 2. Fermeture Transaction
// 1. Fermeture connexion Base de Données
```

## 2.2 Règle n°2 (Piège classique) : Évaluation immédiate des arguments !

L'exécution de la fonction différée a lieu à la fin, **MAIS ses arguments sont évalués immédiatement** au moment où la ligne `defer` est rencontrée !

```Go
func piegeEvaluation() {
	statut := "EN_COURS"

	// PIÈGE : 'statut' est évalué MAINTENANT (vaut "EN_COURS") !
	defer fmt.Println("Log direct :", statut)

	// SOLUTION : Utiliser une closure sans argument qui lira 'statut' à la fin
!
	defer func() {
		fmt.Println("Log via closure :", statut)
	}()

	statut = "SUCCÈS"
}
// Affichage à la sortie :
// Log via closure : SUCCÈS
// Log direct : EN_COURS
```

## 2.3 Pattern avancé : Chronométrage en une ligne (`defer trace()()`)

En combinant l'évaluation immédiate avec une closure retournée, on obtient un pattern très célèbre en Go pour mesurer le temps d'exécution d'une fonction :

```Go
func chronometrer(nomOperation string) func() {
	debut := time.Now() // Évalué immédiatement à l'entrée !
	fmt.Printf("[DÉBUT] %s\n", nomOperation)

	return func() { // Cette fonction retournée sera exécutée par 'defer' à la
sortie !
		fmt.Printf("[FIN] %s terminé en %v\n", nomOperation,
time.Since(debut))
	}
}

func traitementLourd() {
	defer chronometrer("Importation CSV")() // Notez les doubles parenthèses
()() !
	time.Sleep(100 * time.Millisecond)
}
```

*Attention au piège du `defer` dans une boucle `for` :* Comme `defer` ne s'exécute qu'à la fin de la **fonction** (et non à la fin du bloc `{}` de la boucle `for`), faire `defer fichier.Close()` dans une boucle qui ouvre 10 000 fichiers gardera les 10 000 descripteurs ouverts simultanément jusqu'à épuisement du système (`too many open files`). Il faut soit fermer explicitement, soit extraire le corps de la boucle dans une sous-fonction.

---

# 3. Pointeurs et Gestion Mémoire : Stack, Heap et *Escape Analysis*

## 3.1 Les Pointeurs en Go vs C

Comme en C, un pointeur `*T` contient l'adresse mémoire d'une valeur de type `T`.

- L'opérateur **`&`** prend l'adresse mémoire d'une variable (`ptr := &x`).
- L'opérateur `*` déréférence le pointeur pour lire ou modifier la valeur pointée (`*ptr = 42`).
- La *Zero Value* de tout pointeur est **`nil`**. Déréférencer un pointeur `nil` provoque un crash propre du runtime (`panic: runtime error: invalid memory address or nil pointer dereference`).

**Les 2 différences majeures avec le C :**

1. **Zéro arithmétique de pointeurs :** Il est interdit d'additionner ou d'incrémenter un pointeur (`ptr++` ou `ptr + 4` ne compilent pas). Cela garantit la sûreté du typage et du Garbage Collector (impossible de fabriquer un pointeur vers une zone mémoire arbitraire sans passer par le package spécial `unsafe`).
2. **Allocation par référence propre avec `&T{}` ou `new(T)` :** Pas besoin de `malloc(sizeof(T))`.

## 3.2 Stack vs Heap et l'*Escape Analysis* du compilateur

En C :

- Une variable locale (`int x = 10;`) vit sur la **Pile (Stack)**. Si vous retournez `&x`, votre programme a un bug critique (*dangling pointer* / *use-after-free*) car le cadre de pile (*stack frame*) est détruit au `return`.
- Pour qu'une donnée survive à la fonction, vous devez appeler manuellement `malloc()` sur le **Tas (Heap)**, puis penser à appeler `free()`.

**En Go, le développeur ne choisit pas entre Stack et Heap : c'est le compilateur qui décide automatiquement via l'algorithme d'*Escape Analysis* (Analyse d'échappement) !**

```go
// CAS 1 : Allocation sur la STACK (Pile)
// Le pointeur '&x' est passé à 'doubler', mais 'doubler' ne le stocke nulle part ailleurs.
// Le compilateur prouve que 'x' ne survit pas à 'calculLocale' -> Allocation sur la Stack (0 GC) !
func doubler(p *int) {
	*p *= 2
}

func calculLocal() int {
	x := 21
	doubler(&x) // Sharing down (partage vers le bas de la pile) -> Reste sur la Stack !
	return x
}

// CAS 2 : Échappement vers la HEAP (Tas)
// L'adresse de 'y' est retournée à l'appelant. 'y' survit à la fonction 'creerEntier'.
// Le compilateur le détecte : "y escapes to heap" -> Allocation automatique sur la Heap !
func creerEntier() *int {
	y := 42
	return &y // Sharing up (partage vers le haut) -> S'échappe sur la Heap (géré par le GC) !
}
```

### 3.3 Inspecter les décisions du compilateur avec `-gcflags="-m"`

En tant qu'ingénieur cherchant la haute performance en Go, votre objectif est de **minimiser les allocations sur la Heap** pour ne pas donner de travail au Garbage Collector. Go permet d'afficher le rapport exact de l'*Escape Analysis* lors de la compilation grâce au flag `-gcflags="-m"` :

```shell
go build -gcflags="-m" main.go
```

Le compilateur affichera ligne par ligne :

- `moved to heap: y` ou `... escapes to heap` : La variable a dû être allouée sur le tas (coût GC).
- `... does not escape` : Le pointeur reste confiné à la pile (coût zéro, détruit instantanément au `return`).

*Règle de performance en Go (contrairement au réflexe C/C++) :* Ne retournez pas systématiquement des pointeurs `*MaStruct` en pensant optimiser. Retourner un pointeur

force l'allocation sur la **Heap** (*Escape*). Retourner une structure de taille raisonnable **par valeur** (`MaStruct`) permet de tout garder sur la **Stack**, ce qui est jusqu'à 10 fois plus rapide sur un processeur moderne !